

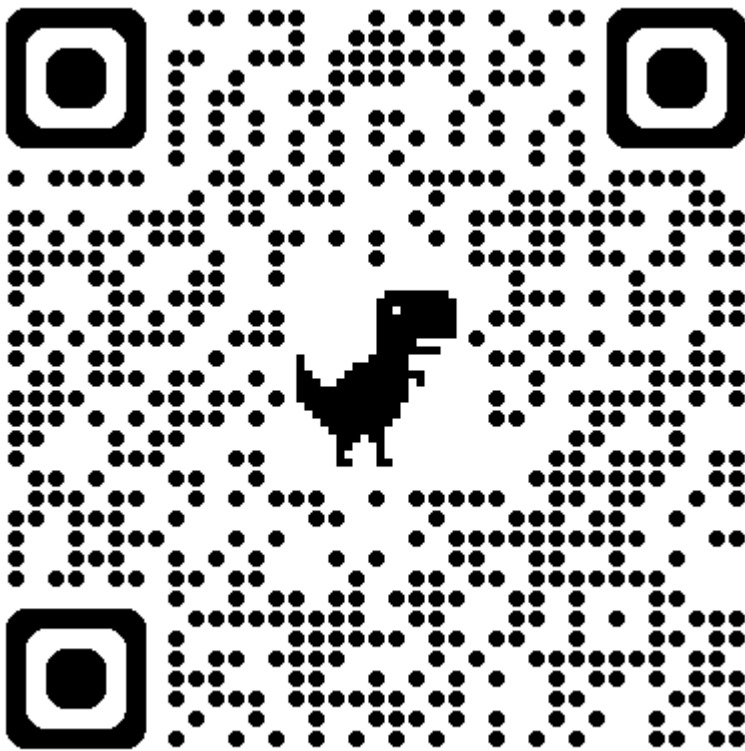
Introduction to Scientific Computing

ENGR1050 – Lecture 7

Fall 2025

Nat Trask

University of Pennsylvania



Introduction to Classes

Basic Concepts



Dictionaries

Alternatives to lists

In-class exercise

Practice what we've seen

Clearing up “markdown” requirement for HWs

Pretend you are tasked with figuring out what this code is supposed to do:

```
s=[['Alice',80,90,75],['Bob',60,55,70],['Cara',95,100,92]]
r=[]
for x in s:
    a=sum(x[1:])/len(x[1:])
    if a>70:
        r.append((x[0],a,'P'))
    else:
        r.append((x[0],a,'F'))
print(r)
```

[] Python

Clearing up “markdown” requirement for HWs

Now think of this one:

Grade summary

Purpose: compute each student's average and mark Pass/Fail. Inputs: list of [name, score1, score2, ...].

Output: printed list of (name, average, 'P' or 'F').

Run: place the Python cell below after this markdown and execute both cells.

```
def grade_summary(students, pass_mark=70):
    """Return list of (name, average, 'P'/'F')."""
    results = []
    for student in students:
        name = student[0]
        scores = student[1:]
        avg = sum(scores) / len(scores)      # compute average score
        status = 'P' if avg >= pass_mark else 'F'
        results.append((name, avg, status))
    return results

students = [
    ['Alice', 80, 90, 75],
    ['Bob', 60, 55, 70],
    ['Cara', 95, 100, 92],
]

print(grade_summary(students))
```

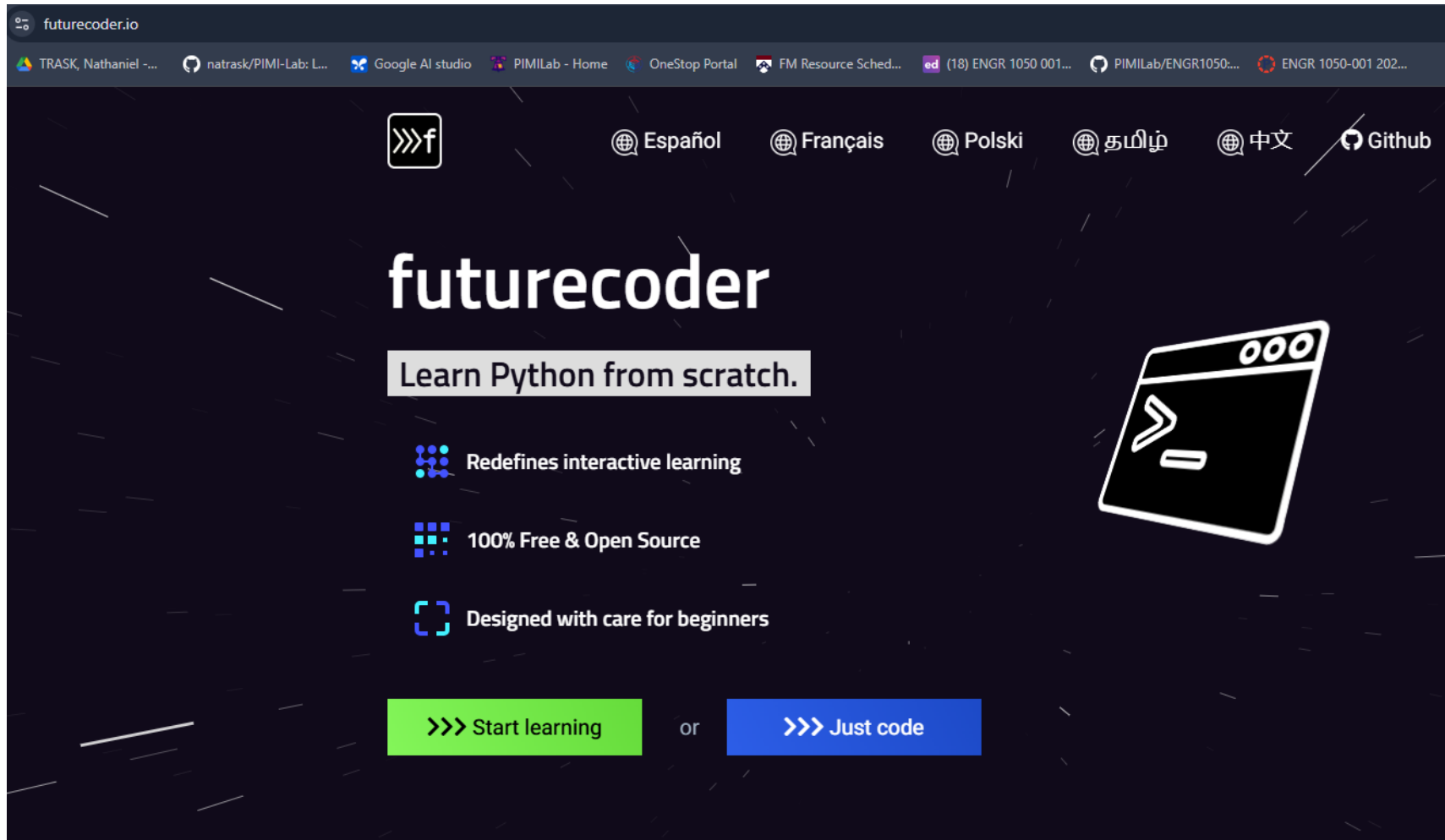
Python

Generate

+ Code

+ Markdown

Preparing for exam



futurecoder.io

LearnPython.org

A reminder: It is very easy for us to tell if you are copying and pasting AI code

```
nums = [10, 20, 30, 40, 50]
total = 0
for n in nums:
    total += n
avg = total / len(nums)
print(avg)
```

We have asked you to not use AI during this first unit of the course – you *must* learn how to think about programs before you automate making them

We are tracking who is using AI and will keep that in mind when we evaluate your progress

```
# Function to calculate the average of a list of numbers in Python
from typing import List

def calculate_average(numbers: List[float]) -> float:
    """
    Calculate the arithmetic mean of a list of numbers.
    :param numbers: List of integers or floats
    :return: Average value as float
    """
    if not numbers:
        raise ValueError("The input list cannot be empty.")
    total = sum(numbers) # sum() adds up all elements
    count = len(numbers) # len() gives the number of elements
    average = total / count
    return average

# Example usage
if __name__ == "__main__":
    sample_numbers = [10, 20, 30, 40, 50]
    print("The average is:", calculate_average(sample_numbers))
```

Last time

Functions

Today: final python building blocks

Dictionaries: AKA fancy lists

Classes

Key concept: encapsulation

Why use functions?

Task: Write code to take the diameters of three balls given in inches and print out their diameters in centimeters

```
diameter_large = 2.54 * 1.65
print('Large ball: ', diameter_large, ' centimeters')

diameter_med = 2.54 * 1.01
print('Medium ball: ', diameter_med, ' centimeters')

diameter_small = 2.54 * 0.46
print('Small ball: ', diameter_small, ' centimeters')
```

Functions let you reuse code and avoid errors from repetition

BEFORE

```
diameter_large = 2.54 * 1.65
print('Large ball: ', diameter_large, 'cm')

diameter_med = 2.54 * 1.01
print('Medium ball: ', diameter_med, 'cm')

diameter_small = 2.54 + 0.46
print('Small ball: ', diameter_small, 'cm')
```

A potential
typo during
rewriting

AFTER

```
def print_as_cm(inches, name):
    cm = 2.54 * inches
    print(name, ': ', cm, 'cm')

print_as_cm(1.65, 'Large ball')
print_as_cm(1.01, 'Medium ball')
print_as_cm(0.46, 'Small ball')
```

only type
these lines
once -
can't make a
mistake!

Functions let you consistently make changes with minimal effort

BEFORE

```
def print_as_cm(inches, name):  
    cm = 2.54 * inches  
    print(name, ':', cm, 'centimeters')  
  
print_as_cm(1.65, 'Large ball')  
print_as_cm(1.01, 'Medium ball')  
print_as_cm(0.46, 'Small ball')
```

AFTER

```
def print_as_cm(inches, name):  
    cm = 2.54 * inches  
    print(name, ':', cm, 'cm')  
  
print_as_cm(1.65, 'Large ball')  
print_as_cm(1.01, 'Medium ball')  
print_as_cm(0.46, 'Small ball')
```

Can change
from
'centimeters'
with only one
change



Functions *encapsulate* functionality and make your code easy to interpret

BEFORE

```
diameter_large = 2.54 * 1.65
print('Large ball: ', diameter_large, 'cm')

diameter_med = 2.54 * 1.01
print('Medium ball: ', diameter_med, 'cm')

diameter_small = 2.54 + 0.46
print('Small ball: ', diameter_small, 'cm')
```

}

}

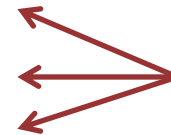
}

What are we
doing here?

AFTER

```
def print_as_cm(inches, name):
    cm = 2.54 * inches
    print(name, ': ', cm, 'cm')

print_as_cm(1.65, 'Large ball')
print_as_cm(1.01, 'Medium ball')
print_as_cm(0.46, 'Small ball')
```



Oh, printing as
centimeters!

Dictionaries

AKA fancy lists

(overview in Jupyter notebook)

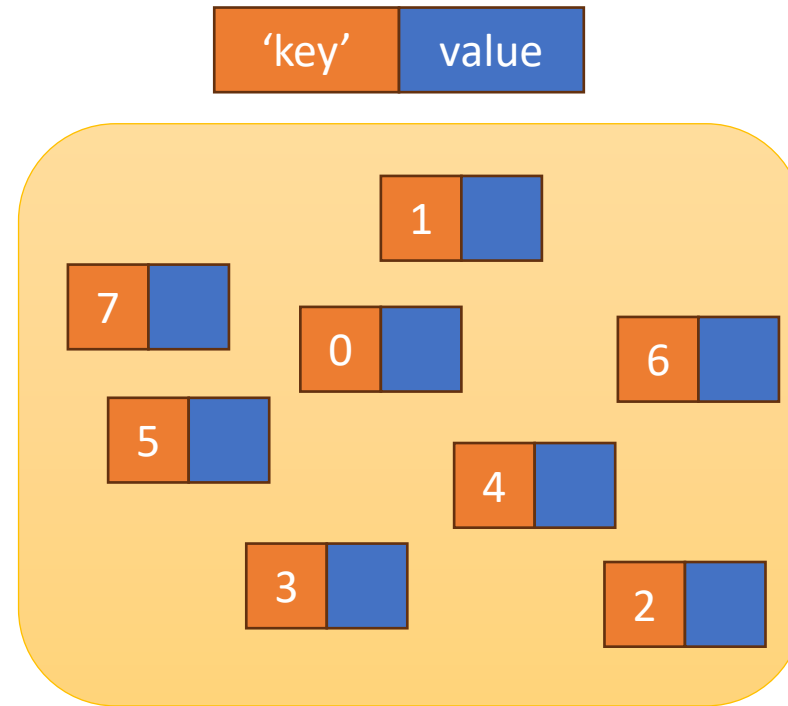
Lists

$O(N)$ lookup

0	
1	
2	
3	
4	
5	
6	
7	

Dictionaries

$O(1)$ lookup



Classes

A tool for encapsulation

A **class** is a Python object which encapsulates a collection of functions and variables to help structure and organize your code.

```
class MyClass:
    """Docstring"""
    classVariable = value

    def classFunction():
        # code to evaluate function
        return functionAnswer
```

Ex:

```
class MyClass:
    """A simple example class"""
    i = 12345

    def f():
        return 'hello world'
```



The screenshot shows a Jupyter Notebook interface. At the top, there is a code cell with the following code: `print(MyClass.i)` and `MyClass.f()`. Below the code cell, the output is displayed. The first line of output is `12345`, which corresponds to `print(MyClass.i)`. The second line of output is `'hello world'`, which corresponds to `MyClass.f()`. The notebook interface includes a play button icon, a checkmark, and a timer showing `0.0s`. The language is set to `Python`.

We'll think about three kinds of classes today

Type 1

Containers for
functions/variables

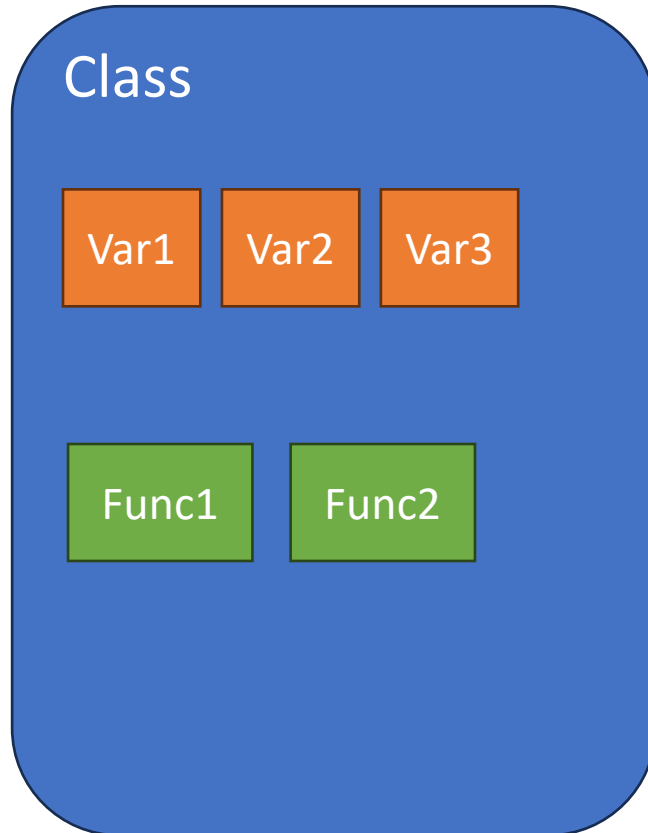
Type 2

Instantiation to
different types

Type 3

Inheritance for
taxonomies

Type 1: Classes group together several variables and functions



```
class MyClass:
    ... """A simple example class"""
    ... i = 12345
    ...
    ... def f():
    ...     ... return "hello world"
```

Python



```
print(MyClass.i)
MyClass.f()
```

[11]



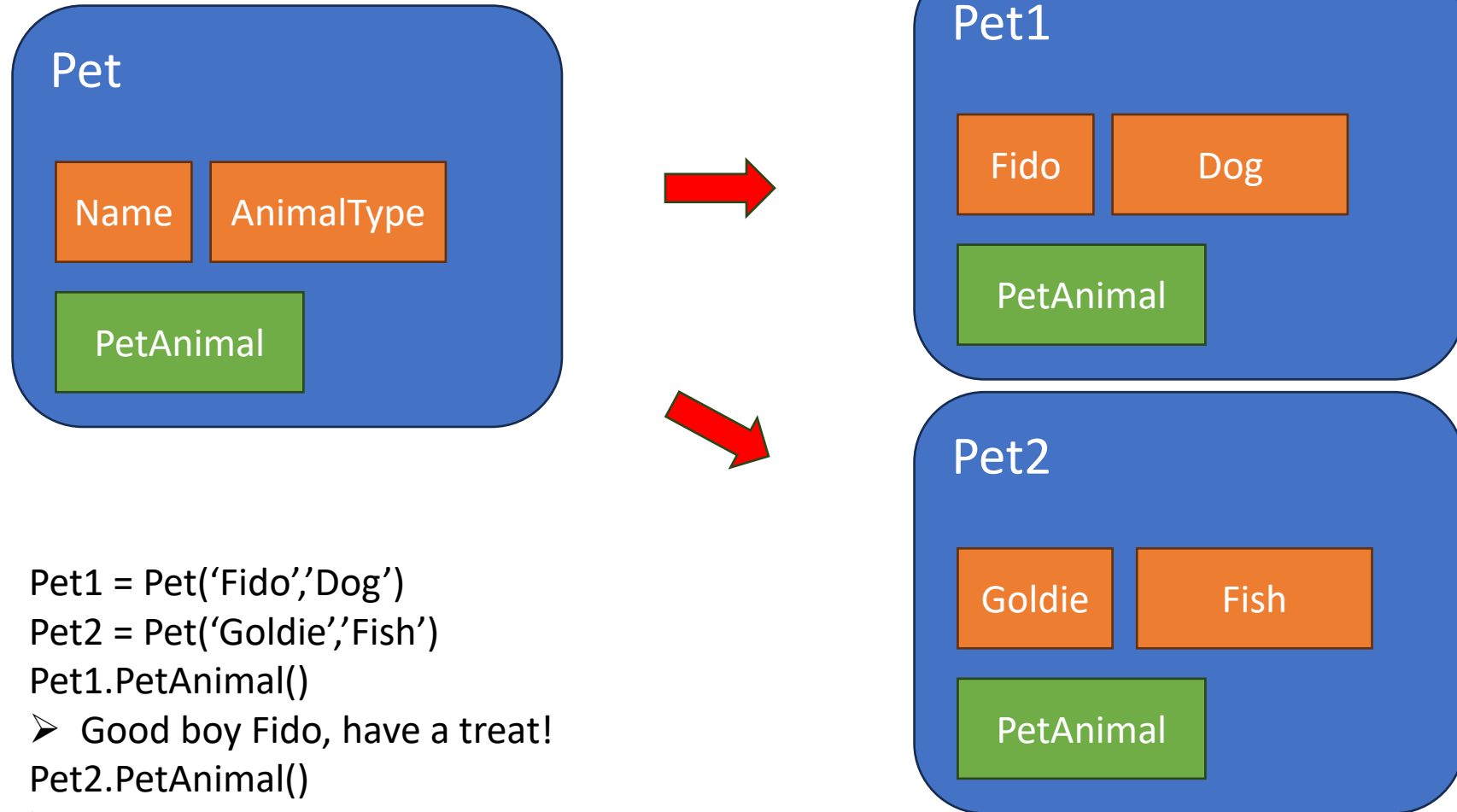
0.0s

Python

... 12345

... 'hello world'

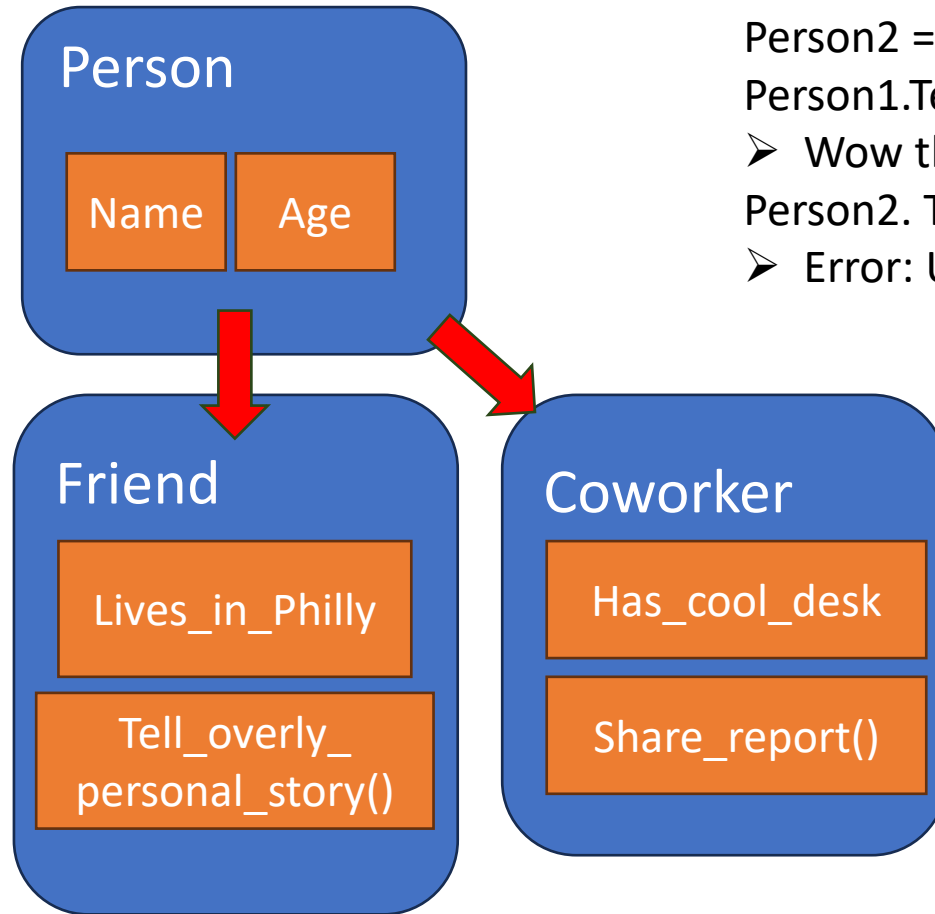
Type 2: Class instantiation



```
Pet1 = Pet('Fido','Dog')  
Pet2 = Pet('Goldie','Fish')  
Pet1.PetAnimal()  
➤ Good boy Fido, have a treat!  
Pet2.PetAnimal()  
➤ My hand got wet trying to pet Goldie.
```

Allows specialization of behavior to different contexts

Type 3: *Inheritance* lets us build up specialized versions of generic classes



```
Person1 = Friend('Bob',25,False)
Person2 = Coworker('Nancy',26,True)
Person1.Tell_overly_personal_story()
➤ Wow that was embarrassing, but funny!
Person2. Tell_overly_personal_story()
➤ Error: Undefined, why would you do that?
```

Think of this like a taxonomy: rectangles are examples of shapes, dogs are examples of animals, and so on

Syntax for types 1-2:

```
class className:
    classvar1 = value # all members of class share this
    ...
    classvarN = value # all members of class share this

    def __init__(self, input1, ..., inputN):
        self.var1 = input1 # specialized var
        ...
        self.varN = inputN # specialized var

    def myfunc(self, func_input):
        <statement>
```

- The `__init__` function is called automatically when the class is instantiated
- All members of the class share the same `classvar1-N` variables
- The `self.var1-N` variables are specialized to each class. `self` denotes this

Syntax

```
class Dog:
    def __init__(self, name, age, breed):
        self.name = name
        self.age = age
        self.breed = breed

    def whatKindofAnimal():
        return f"I am a dog!"

    def bark(self):
        return f"{self.name} says woof!"

    def get_info(self):
        return f"Name: {self.name}, Age: {self.age}, Breed: {self.breed}"
```

Specify parent class

Specify how to initialize
parent variables

```
class WorkingDog(Dog):
    def __init__(self, name, age, breed, job):
        super().__init__(name, age, breed)
        self.job = job

    def do_job(self):
        return f"{self.name} is doing their job: {self.job}"

# Example usage:
working_dog = WorkingDog("Max", 4, "German Shepherd", "Police Dog")
print(working_dog.bark())
print(working_dog.get_info())
print(working_dog.do_job())
```

Python